

Lab 5 Extra

准备工作

请在自动初始化分支后，在开发机依次执行：

```
$ cd ~/学号
$ git fetch
$ git checkout lab5-extra
```

初始化的 `lab5-extra` 分支基于你课下完成的 `lab5` 分支，并且在 `tests` 目录下添加了 `lab5_verity` 样例测试目录。

题目背景 & 功能概览

完成 Lab5 后，MOS 已经能够通过文件系统服务进程管理普通文件的打开、映射、读写、截断和写回。本题在此基础上加入一个简化的 Verity（完整性校验）机制：当一个普通文件被 seal（密封）后，文件系统需要把它视为只读文件，并在之后的访问过程中校验其内容是否仍与 seal 时一致。

本题需要支持以下功能：

1. 对普通文件调用 `fseal(path)`，记录该普通文件当前的大小和全文件摘要，并把它标记为 sealed（被密封）。
2. 对 sealed 普通文件调用 `fverify(path)`，重新计算摘要并与保存值比较，不一致返回 `-E_VERIFY`。
3. sealed 文件不能再通过正常写路径修改。使用 `O_WRONLY`、`O_RDWR` 或带 `O_TRUNC` 打开 sealed 文件时，应返回 `-E_SEALED`。
4. 使用 `O_RDONLY` 打开 sealed 文件时，应先进行完整性校验。

本题还提供了 `fs_debug_corrupt()` 作为测试辅助接口。它用于绕过正常写路径篡改数据块，帮助检查校验逻辑是否能发现内容变化。

本题新增的错误码含义如下：

1. `-E_VERIFY`：sealed 文件摘要校验失败。
2. `-E_SEALED`：试图通过正常写路径修改 sealed 文件。

实现思路

1. Verity 核心实现

在 `include/error.h` 中引入本题所需错误码：

```
// File not a valid executable
#define E_NOT_EXEC 13

// ----- 新增开始 -----

// Verity digest mismatch on a sealed file
#define E_VERIFY 14

// Attempt to write to a sealed file
#define E_SEALED 15

// ----- 新增结束 -----
```

在 `user/include/fs.h` 中引入本题所需数据结构：

```
struct File {
    char f_name[MAXNAMELEN];
    uint32_t f_size;
    uint32_t f_type;
    uint32_t f_direct[NDIRECT];
    uint32_t f_indirect;
    struct File *f_dir;
    char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (3 + NDIRECT) * 4 - sizeof(void
*)];
} __attribute__((aligned(4), packed));

// ----- 新增开始 -----

#define FVERITY_MAGIC 0x56545931
#define FVERITY_SEALED 0x1

struct FileVerity {
    uint32_t v_magic; // verity 数据魔数
    uint32_t v_flags; // sealed 状态标记
    uint32_t v_size; // seal 时记录的文件大小
    uint32_t v_digest; // seal 时记录的全文件摘要
};

static inline struct FileVerity *file_verity(struct File *f) {
    return (struct FileVerity *)f->f_pad;
}

static inline int file_is_sealed(struct File *f) {
    struct FileVerity *v = file_verity(f);
    return v->v_magic == FVERITY_MAGIC && (v->v_flags & FVERITY_SEALED);
}

// ----- 新增结束 -----

#define FILE2BLK (BLOCK_SIZE / sizeof(struct File))
```

注意：

1. `FileVerity` 存放在 `struct File` 末尾的 `f_pad` 中，不改变 `FILE_STRUCT_SIZE`。

2. `v_magic` 当值为 `FVERITY_MAGIC` 时, 说明 `f_pad` 中保存的是有效的 verity 数据。
3. 当 `v_flags` 包含 `FVERITY_SEALED` 时表示文件已 seal。
4. `v_size` 记录 seal 时的文件大小, 用于在校验时发现文件大小变化。
5. `v_digest` 记录 seal 时的全文件摘要, 用于之后的完整性校验。
6. 你可以调用 `file_verity()` 函数来得到该文件的 `FileVerity` 结构体。
7. 你可以调用 `file_is_sealed()` 函数来判断一个文件是否 sealed。

在 `fs/verity_core.c` 中补全 Verity 机制核心实现:

1. 在 `file_seal()` 中为普通文件建立 verity 数据。

```
if (!file_regular(f)) {
    return -E_INVALID;
}

// Lab5-Extra: Your code here. (1/6)
// 调用 file_digest() 计算当前文件的全文件摘要。
// 然后填写该文件的 FileVerity 结构体,
// 需要填写的字段有: v_magic、v_flags、v_size 和 v_digest。

// Lab5-Extra: End (1/6).

file_close(f);
```

注意:

- `int file_digest(struct File *f, uint32_t *hash_store)` 用于计算文件 `f` 当前内容的全文件摘要。调用成功时返回 `0`, 并把摘要写入第二个参数指向的变量; 调用失败时返回对应错误码。你可以直接调用该函数来计算当前文件的全文件摘要。
 - 可调用 `file_verity(f)` 得到该文件的 `FileVerity` 结构体;
 - `v_size` 应填写为当前文件大小 `f_size`;
 - 若 `file_digest()` 返回错误, 应直接返回该错误码。
2. 在 `file_verify()` 中校验 sealed 文件内容是否仍与 seal 时一致。

```
if (f->f_size != v->v_size) {
    return -E_VERIFY;
}

// Lab5-Extra: Your code here. (2/6)
// 调用 file_digest() 重新计算摘要, 并与 v->v_digest 比较。

// Lab5-Extra: End (2/6).

return 0;
```

注意:

- 若重新计算摘要失败, 应直接返回该错误码;
- 若重新计算出的摘要与 `v->v_digest` 不一致, 返回 `-E_VERIFY`;

- 校验通过时返回 0。

2. 用户态接口与文件系统请求

本题需要把三个新用户态请求接入文件系统服务端。整体链路如下：

```
fseal(path) -> fsipc_seal(path) -> FSREQ_SEAL -> serve_seal() -> file_seal()
fverify(path) -> fsipc_verify(path) -> FSREQ_VERIFY -> serve_verify() ->
file_verify()
fs_debug_corrupt(...) -> fsipc_debug_corrupt(...) -> FSREQ_DEBUG_CORRUPT ->
serve_debug_corrupt() -> file_debug_corrupt()
```

下面以 `fs_debug_corrupt(...) -> fsipc_debug_corrupt(...) -> FSREQ_DEBUG_CORRUPT -> serve_debug_corrupt() -> file_debug_corrupt()` 这条链路为示例进行接入，另外两条 `seal / verify` 链路请在后文参照它自行完成。

2.1 注册请求号

在 `user/include/fsreq.h` 中引入 `FSREQ_DEBUG_CORRUPT` 请求号：

```
FSREQ_REMOVE,
FSREQ_SYNC,
// ----- 新增开始 -----
FSREQ_DEBUG_CORRUPT,
// ----- 新增结束 -----
MAX_FSREQNO,
};
```

注意新引入的请求号要在 `MAX_FSREQNO` 之前。

2.2 定义请求结构体

然后加入所需的请求结构体：

```
MAX_FSREQNO,
};

// ----- 新增开始 -----

struct Fsreq_debug_corrupt {
    char req_path[MAXPATHLEN];
    u_int req_block_index;
    u_int req_byte_offset;
    u_int req_value;
};

// ----- 新增结束 -----

struct Fsreq_open {
    char req_path[MAXPATHLEN];
    u_int req_omode;
};
```

2.3 声明用户态函数

在 `user/include/lib.h` 中加入所需函数声明：

先引入新增的 `fsipc` 函数声明：

```
int fsipc_sync(void);
int fsipc_incref(u_int);
// ----- 新增开始 -----
int fsipc_debug_corrupt(const char *path, u_int block_index, u_int byte_offset,
u_int value);
// ----- 新增结束 -----
```

再引入所需的用户接口函数声明：

```
int ftruncate(int fd, u_int size);
int sync(void);
// ----- 新增开始 -----
int fs_debug_corrupt(const char *path, u_int block_index, u_int byte_offset,
u_int value);
// ----- 新增结束 -----
```

2.4 实现用户态 `fsipc` 函数

在 `user/lib/fsipc.c` 中实现函数 `fsipc_debug_corrupt()`：

```
int fsipc_sync(void) {
    return fsipc(FSREQ_SYNC, fsipcbuf, 0, 0);
}

// ----- 新增开始 -----

int fsipc_debug_corrupt(const char *path, u_int block_index, u_int byte_offset,
u_int value) {
    struct Fsreq_debug_corrupt *req;

    if (path == 0 || path[0] == '\0' || strlen(path) >= MAXPATHLEN) {
        return -E_BAD_PATH;
    }

    // 将 fsipcbuf 转为对应的结构体指针。
    req = (struct Fsreq_debug_corrupt *)fsipcbuf;
    // 填写请求结构体中的各个字段。
    strcpy((char *)req->req_path, path);
    req->req_block_index = block_index;
    req->req_byte_offset = byte_offset;
    req->req_value = value;
    // 调用 fsipc() 发送请求号和请求结构体。
    return fsipc(FSREQ_DEBUG_CORRUPT, req, 0, 0);
}

// ----- 新增结束 -----
```

2.5 实现用户接口

在 `user/lib/file.c` 中实现函数 `fs_debug_corrupt()`。该函数作为用户程序直接调用的接口只负责转发参数：

修改后应为：

```
int sync(void) {
    return fsipc_sync();
}

// ----- 新增开始 -----

int fs_debug_corrupt(const char *path, u_int block_index, u_int byte_offset,
u_int value) {
    return fsipc_debug_corrupt(path, block_index, byte_offset, value);
}

// ----- 新增结束 -----
```

2.6 注册服务端处理函数

`fs/verity_serv.c` 已经给出了三条链路的服务端处理函数。只需要在 `fs/serv.c` 的 `serve_table` 中注册好对应的函数即可：

```
void *serve_table[MAX_FSREQNO] = {
    [FSREQ_OPEN] = serve_open,
    [FSREQ_MAP] = serve_map,
    [FSREQ_SET_SIZE] = serve_set_size,
    [FSREQ_CLOSE] = serve_close,
    [FSREQ_DIRTY] = serve_dirty,
    [FSREQ_REMOVE] = serve_remove,
    [FSREQ_SYNC] = serve_sync,
    // ----- 新增开始 -----
    [FSREQ_DEBUG_CORRUPT] = serve_debug_corrupt,
    // ----- 新增结束 -----
};
```

2.7 参照示例完成 seal / verify 链路

请你添加好示例链路后，再参照示例，将 `fseal(path)` 和 `fverify(path)` 两条链路也接入 IPC，这两条链路只需要传递 `path` 参数，所以共用 `struct Fsreq_path` 请求结构体。

接入这两条链路时用到的信息如下：

```
// 请求号
FSREQ_SEAL,
FSREQ_VERIFY,

// 请求结构体
struct Fsreq_path {
    // Lab5-Extra: Your code here. (3/6)
    // 只需要 req_path 参数。
```

```

    // Lab5-Extra: End (3/6).
};

// 函数声明
// fsipc 函数
int fsipc_seal(const char *path);
int fsipc_verify(const char *path);
// 用户接口函数
int fseal(const char *path);
int fverify(const char *path);

// 你可使用以下框架实现相关 fsipc 函数
int fsipc_seal(const char *path) {
    struct Fsreq_path *req;

    if (path == 0 || path[0] == '\0' || strlen(path) >= MAXPATHLEN) {
        return -E_BAD_PATH;
    }

    // Lab5-Extra: Your code here. (4/6)
    // 将 fsipcbuf 转为对应的结构体指针, 填写 req_path, 并调用 fsipc() 发送对应请求。

    // Lab5-Extra: End (4/6).
}

int fsipc_verify(const char *path) {
    struct Fsreq_path *req;

    if (path == 0 || path[0] == '\0' || strlen(path) >= MAXPATHLEN) {
        return -E_BAD_PATH;
    }

    // Lab5-Extra: Your code here. (5/6)
    // 将 fsipcbuf 转为对应的结构体指针, 填写 req_path, 并调用 fsipc() 发送对应请求。

    // Lab5-Extra: End (5/6).
}

```

此处给出一个 TODO list 方便同学们对照：

- ☐ 注册请求号
- ☐ 新增请求结构体
- ☐ 声明用户态函数, 包括 `fsipc` 与接口函数
- ☐ 实现 `fsipc` 接口函数
- ☐ 自行实现用户态接口函数
- ☐ 在 `serve_table` 中注册好对应的 `serve` 函数 (`serve_seal` 与 `serve_verify`)

3. 在 `serve_open()` 中加入 sealed 文件的打开检查

先在 `fs/serv.c` 文件顶部引入 `fs/verity_internal.h`：

```
#include "serv.h"
// ----- 新增开始 -----
#include "verity_internal.h"
// ----- 新增结束 -----
#include <fd.h>
```

然后在 `serve_open()` 中加入 sealed 文件的打开检查，位置在保存文件指针之后、执行 `O_TRUNC` 之前：

```
// Save the file pointer.
o->o_file = f;

// Lab5-Extra: Your code here. (6/6)
// 若 f 已 sealed:
// 1. 遇到 O_TRUNC、O_WRONLY 或 O_RDWR，向请求进程返回 -E_SEALED 并 return;
// 2. 只读打开时调用 file_verify(f)，若返回错误则把该错误返回给请求进程并 return。

// Lab5-Extra: End (6/6).

// If mode include O_TRUNC, set the file size to 0
if (rq->req_omode & O_TRUNC) {
```

注意：

- 你可以调用 `file_is_sealed()` 函数来判断一个文件是否 sealed。
- 你可以使用 `(rq->req_omode & O_ACCMODE)` 判断文件打开模式。如果你对文件打开模式不熟悉，可以查阅 `user/include/lib.h`。

样例输出 & 本地测试

对于如下用户程序样例：

```
#include <error.h>
#include <lib.h>
#include <string.h>

static void expect_eq(int got, int want, const char *step) {
    if (got != want) {
        user_panic("case1 sample: %s returned %d, expected %d", step, got, want);
    }
}

static void expect_nonneg(int got, const char *step) {
    if (got < 0) {
        user_panic("case1 sample: %s returned %d, expected non-negative", step, got);
    }
}

static void finish(void) {
    int r;
    syscall_read_dev(&r, 0x10000010, 4);
}
```



```

static void create_text_file(const char *path, const char *data) {
    int fd, r;
    u_int len;

    fd = open(path, O_CREAT | O_RDWR | O_TRUNC);
    expect_nonneg(fd, "open file for create");
    len = strlen(data);
    if (len != 0) {
        r = write(fd, data, len);
        expect_eq(r, len, "write initial content");
    }
    expect_eq(close(fd), 0, "close created file");
}

static void check_text_file(const char *path, const char *data) {
    char buf[64];
    int fd, r;
    u_int len;

    fd = open(path, O_RDONLY);
    expect_nonneg(fd, "open sealed file read-only");
    memset(buf, 0, sizeof(buf));
    len = strlen(data);
    r = read(fd, buf, len);
    expect_eq(r, len, "read sealed file content");
    if (strcmp(buf, data) != 0) {
        user_panic("case1 sample: sealed file content mismatch");
    }
    expect_eq(close(fd), 0, "close sealed file after read");
}

int main(void) {
    char buf[8];
    int fd;
    const char *msg = "hello verity";

    debugf("verity 1 begin\n");

    create_text_file("/v1_empty", "");
    expect_eq(fseal("/v1_empty"), 0, "fseal empty file");
    expect_eq(fverify("/v1_empty"), 0, "fverify empty file");
    fd = open("/v1_empty", O_RDONLY);
    expect_nonneg(fd, "open sealed empty file");
    expect_eq(read(fd, buf, sizeof(buf)), 0, "read sealed empty file");
    expect_eq(close(fd), 0, "close sealed empty file");
    debugf("verity 1 empty seal verify ok\n");

    create_text_file("/v1_short", msg);
    expect_eq(fseal("/v1_short"), 0, "fseal short file");
    expect_eq(fverify("/v1_short"), 0, "fverify short file");
    expect_eq(fseal("/v1_short"), 0, "repeat fseal short file");
    expect_eq(fverify("/v1_short"), 0, "fverify short file after repeat seal");
    debugf("verity 1 short seal verify ok\n");

    check_text_file("/v1_short", msg);

```

```
debugf("verity 1 readonly content ok\n");

debugf("verity 1 ok\n");
finish();
return 0;
}
```

应当输出以下关键内容：

```
verity 1 begin
verity 1 empty seal verify ok
verity 1 short seal verify ok
verity 1 readonly content ok
verity 1 ok
```

样例说明

1. 样例创建空文件 `/v1_empty`，调用 `fseal()` 生成摘要信息，再调用 `fverify()` 校验，随后以只读方式打开、读取并关闭该 sealed 文件。
2. 样例创建短文件 `/v1_short`，写入 `hello verity`，检查短文件的 seal/verify、重复 seal、只读打开、读取内容和关闭流程。
3. 该样例同时也是提交评测中的第 1 个测试点，`make test lab=5_verity` 会运行与该测试点一致的本地样例。

你可以使用：

- `make test lab=5_verity && make run` 在本地测试上述样例（调试模式）
- `MOS_PROFILE=release make test lab=5_verity && make run` 在本地测试上述样例（开启优化）

提交评测

```
$ cd ~/学号
$ git add -A
$ git commit -m "message"
$ git push
```

在线评测时，`.mk` 文件、`Makefile` 文件、`init/init.c`、`tests/` 和 `tools/` 目录可能被替换为标准版本，请不要把实际功能逻辑写在这些文件中。

评测标准

测试点序号	评测内容	分数
1	样例：空文件和短文件 seal/verify	15
2	接入新增IPC请求链路	20
3	Verity 核心实现	20
4	sealed 文件写保护和 open 校验	25

测试点序号	评测内容	分数
5	综合测试	20

测试点依赖关系如下（箭头方向表示依赖方向）：

